Greedy Algorithm

Nattee Niparnan

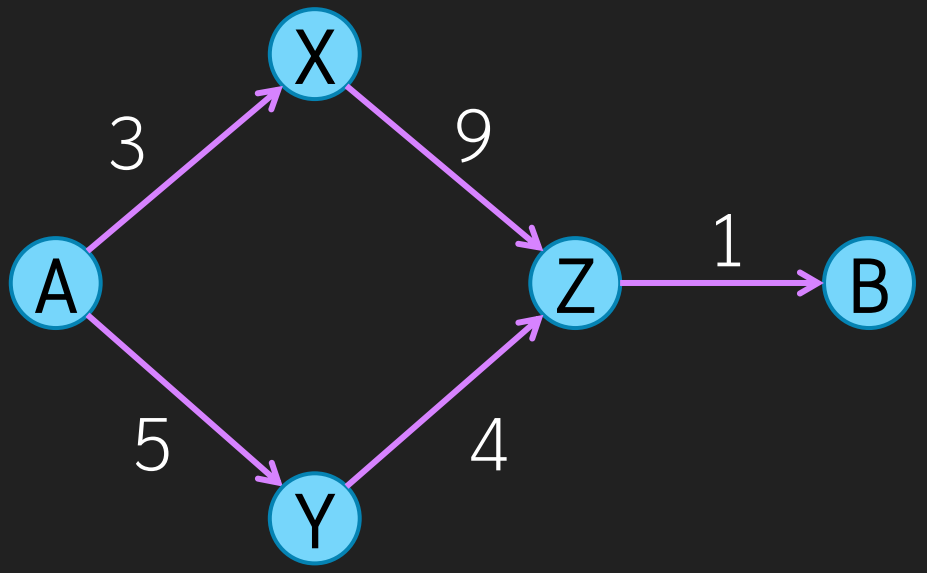
Department of Computer Engineering
Chulalongkorn University

Overview

- Greedy is another algorithm design framework
  - Greedy works correctly on just some problem only
- Key Idea:
  - Just like Brute Force, Greedy consider constructing a candidate solution
  - However, it does not aim to enumerate all candidate solutions, it just want to create the correct one.
  - Construction of the solution is divided into several steps. At each step, greedy algorithm pick the best option of this step only without trying any other option or looking ahead into another step
    - How to define best depends on the algorithm
    - Because it does not try another option (or do any planning ahead), greedy is very fast
    - Most of the time, the best options DOES NOT guarantee a correct solution, but some problem can be solve by greedy

Example of Greedy That is Wrong

- Want a fastest way to drive from point A to B.
  - Each edge is a road and the label on the edge is how long to go through the that road
  - Brute force is to try all possible routes from A to B and pick the shortest routes
    - There are two routes, A->X->Z->B and A->Y->Z->B and the second route is better
  - Greedy is that at any node, we will pick the fastest edge going out of that node without consider the overall planning
    - Start at A, we will go to X because A->X is faster than A->Y
    - This is wrong
- Greedy is not correct on every problem



Greedy needs proof

- Greedy is not guaranteed to be correct.
  - It just pick the best option at each step
    - The best option at that step might not be the
- We need to proof that the greedy actually work for our problem
- This chapter consider two example problems and prove that a greedy method works for each problem

Rational Knapsack

Variation of 01-Knapsack

Rational Knapsack

- The problem is the same as 01-Knapsack except that we can pick fraction of each object
- Input:
  - A number *W*, the capacity of the sack
  - n values of weight and price
    - w_i = weight of the i^{th} items
    - p_i = price of the i^{th} item
- Output:
  - x_i such that
    - $0 \leq x_i \leq 1$
    - $\sum x_i p_i$ is maximum
    - $\sum x_i w_i \leq W$

Xi indicates how much of item #i we will take
X = 0 means take non
X = 1 means take whole
X=0.2 means take only 1/5

Compare to the output of 0-1 Knapsack

Output:

- A subset *S* of {1,2,3,...,n} such that
  - $\sum_{i \in S} p_i$ is maximum
  - $\sum_{i \in S} w_i \leq W$

Recall the brute force

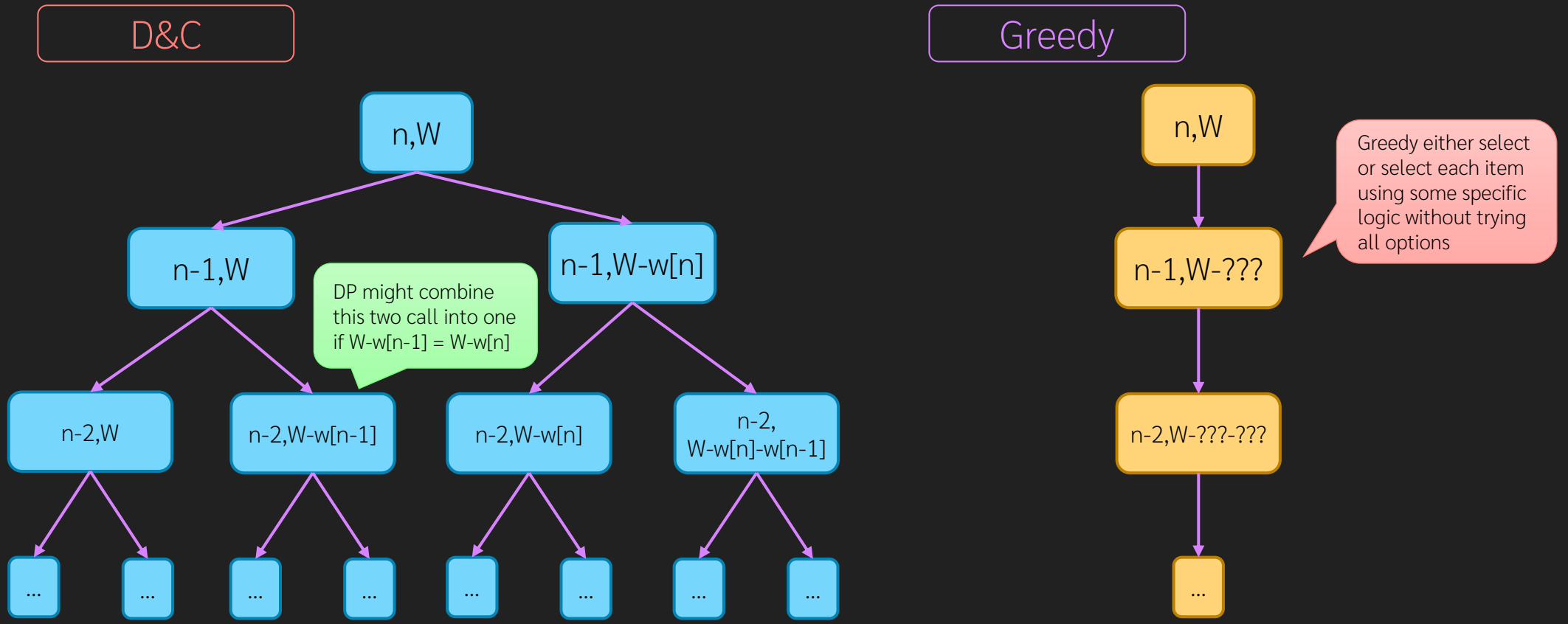
- 01-Knapsack can be solved by brute force simply by formulating the problem as a Constraint Optimization Problem.
  - Solve by enumerate every combination of items and find the one that the sum of weight does not exceed W with maximum sum of price
  - There are $O(2^n)$ candidate solutions
  - For each item we have two possible choices, take it or leave it and we must try all combinations
- Dynamic Programming just reformulate the problem as D&C and recognize duplicate subproblem

```
def knapsack(idx,pick,remain,sum_value)
  if (idx == 0)
    if sum_value > max
      max = sum_value
      best_answer = pick
    return

  # We don't pick item #idx
  pick[idx] = false
  knapsack(idx-1,pick,remain,sum_value)

  if (remain >= w[idx])
    # We pick item #idx only when possible
    pick[idx] = true
    knapsack(idx-1,pick,remain -
             w[idx],sum_value + p[idx])
  end
end

# start from n
knapsack(n,[false,…,false],W,0)
```

```
# D&C version where knapsack(a,b) is the best total price
# we can get when consider only item 1..idx
# and the remaining weight is *remain*
def knapsack(idx,remain)
  if (idx == 0 || remain == 0)
    return 0
  if (remain >= w[idx])
    r1 = knapsack(idx-1,remain)
    r2 = knapsack(idx-1,remain - w[idx]) + p[idx]
    return max(r1,r2)
  else
    return knapsack(idx-1,remain)
end
```

Greedy Approach

- At each step we will do just only one choice



Greedy for Rational Knapsack

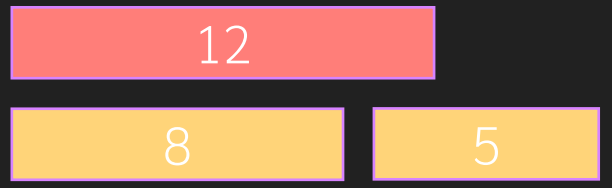
- For Rational Knapsack, we take the most precious object first, by precious we mean best price/weight ratio
  - The items will be sorted first by price/weight
- If we cannot pick it as a whole because of too much weight, just pick fraction of it as much as possible
- This idea is not work for 0-1 Knapsack because we might have to pick less precious one to fill the remaining space
- This is O(n log n) from sorting
  - The actual selection is O(n)

```
def rational_knapsack(idx,pick[1..n],remain,sum_value)
  if (idx == 0 || remain == 0)
    # no need to check if it is more than max
    # we will prove that this is the solution
    best_answer = pick
    max = sum_value
    return
  if (remain >= w[idx])
    pick[idx] = 1
    knapsack(idx-1,pick,remain-w[idx],sum_value+p[idx])
  else
    pick[idx] = remain / w[idx]
    knapsack(idx-1,pick,0,sum_value+p[idx]*pick[idx])
  end
end

items = []
for i from 1 to n
  # build array of [ratio, price, weight]
  items.append([p[i]/w[i],p[i],w[i]])
sort(items)
for i from 1 to n
  # assign back to p and w
  p[i] = items[i][2];
  w[i] = items[i][3];

rational_knapsack(n,[0,...,0],W,0)
```

Items is sorted from least precious to most, so we start from the back

Example

- W = 5
- 3 items
  - #1 Weight 4, price 12 (ratio = 3)
  - #2 Weight 3, price 8 (ratio = 2.66)
  - #3 Weight 2, price 5 (ratio = 2.5)



- For 01-Knapsack it is better to choose #2 and #3 even though they are less precious than #1
- For Rational Knapsack, the best solution is to take #1 as a whole and take 1/3 of #2

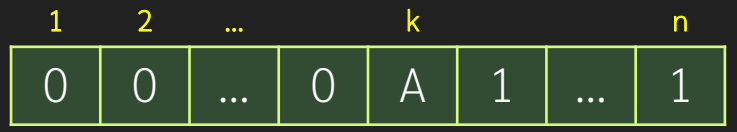
Why Greedy Work?

- Again, a Greedy approach is not guaranteed to be correct in every problem
- For rational knapsack, we must prove that our greedy produces correct solution
- Let x[1..n] be the solution from our greedy approach
  - x is the `pick[1..n]` in the code
- Let y[1..n] be the optimal solution (we don't know what y[] is but we know that it exists)
  - We will prove that
    - $\sum x[i]w[i] \leq W$ (selected item does not exceed capacity)
    - $\sum x[i]p[i] \geq \sum y[i]p[i]$ (our solution has the same value as the best solution)

Proof of Greedy Rational Knapsack

- Obvious from the code that $\sum x[i]w[i] = W$
- Obvious that x is sorted
  - Start with 0, ends with 1
  - At position k, X[k] might be fractional (X[k] < 1)

```
def rational_knapsack(idx,pick[1..n],remain,sum_value)
  if (idx == 0 || remain == 0)
    # no need to check if it is more than max
    # we will prove that this is the solution
    best_answer = pick
    max = sum_value
    return
  if (remain >= w[idx])
    pick[idx] = 1
    knapsack(idx-1,pick,remain-w[idx],sum_value+p[idx])
  else
    pick[idx] = remain / w[idx]
    knapsack(idx-1,pick,0,sum_value+p[idx]*pick[idx])
  end
end
```



$$\sum x[i]p[i] \geq \sum y[i]p[i]$$
$$\sum x[i]p[i] - \sum y[i]p[i] \geq 0$$
$$\sum (x[i] - y[i])p[i] \geq 0$$

We will use x_i, y_i, w_i and p_i instead of x[i], y[i], w[i] and p[i] to save space

Proof of Greedy Rational Knapsack

$$\sum_{i=1}^{n}(x_i - y_i)p_i = \sum_{i=1}^{k-1}(x_i - y_i)p_i + (x_k - y_k)p_k + \sum_{j=k+1}^{n}(x_j - y_j)p_j$$

$$= \sum_{i=1}^{k-1}(x_i - y_i)w_i\frac{p_i}{w_i} + (x_k - y_k)w_i\frac{p_k}{w_k} + \sum_{j=k+1}^{n}(x_j - y_j)w_j\frac{p_j}{w_j}$$

$$\geq \sum_{i=1}^{k-1}(x_i - y_i)w_i\frac{p_k}{w_k} + (x_k - y_k)w_i\frac{p_k}{w_k} + \sum_{j=k+1}^{n}(x_j - y_j)w_i\frac{p_k}{w_k} = \sum_{i=1}^{n}(x_i - y_i)w_i\frac{p_k}{w_k}$$

i < k thus $p_i/w_i < p_k/w_k$
But $x_i = 0$ hence $(x_i - y_i)$ is negative

j > k and $x_j >= y_j$
thus $p_j/w_j >= p_k/w_k$

$$\sum_{i=1}^{n}(x_i - y_i)p_i \geq \sum_{i=1}^{n}(x_i - y_i)w_i\frac{p_k}{w_k}$$

$$= \frac{p_k}{w_k}\sum_{i=1}^{n}(x_i - y_i)w_i$$

$$= \frac{p_k}{w_k}(\sum(x_i w_i) - \sum(y_i w_i))$$

This fraction is positive

This sum is 1

This sum is at most 1

This term is positive, QED.

Knapsack Summary

- Select most precious item first
- Need to show that our greedy produce correct solution

Activity Selection

Activity Selection Problem

- Problem:
  - There are N jobs, each has been scheduled to start and stop at specific time
  - We must pick as many as possible jobs so that the schedule of selected jobs do not conflict
  - Real world example is when we go to an event with N shows, each has published schedules, and we want to see as many show as possible
- Input:
  - start[1..n] start time of each job
  - stop[1..n] stop time of each job (assume start[i] <= stop[i])
- Output:
  - A sequence S[1..k] of the selected job such that
  - for any pair (p,q) 1 <= p < q <= k, the range start[S[p]]..stop[S[p]] does not intersect with start[S[q]]..stop[S[q]]
  - K is maximum

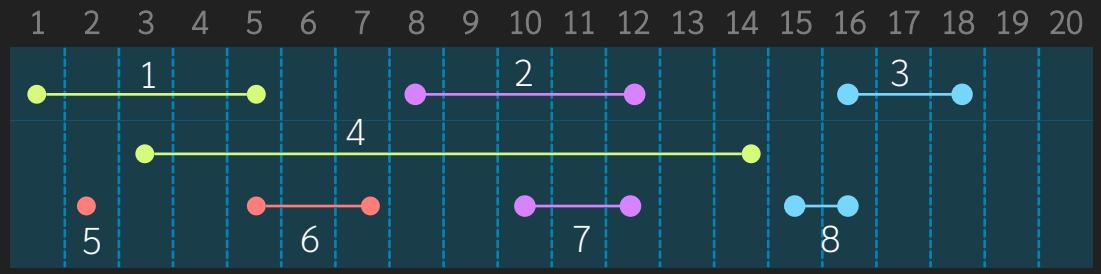
Brute Force Approach

- This is just standard combination problem with optimization.
  - Just try every combination of N job, find largest non-conflicting combination
- For each candidate solution of length k, we must do $O(k^2)$ to check for intersection
  - We can do better by stopping the recursion as soon as the newly selected activity overlap the currently selected ones

Should do this as an exercise

```
def intersect(p,q)
  return start[p] <= stop[q] &&
         stop[p] >= start[q]
end

def activity(idx,sol)
  if idx == N
    conflict = false
    for p from 1 to sol.length
      for q from p+1 to sol.length
        if (p!=q && intersect(sol[p],sol[q]))
          conflict = true
          break
      if conflict
        break;
    if conflict == false && sol.length > max
      max_length = sol.length
      best = sol
  else
    activity(idx+1,sol)
    activity(idx+1,sol.append(idx))
end


max_length = 0
activity(1,[])
```

Example

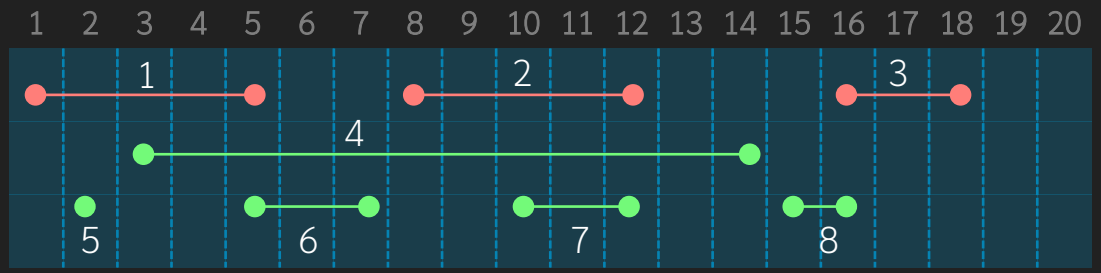


#	start	stop
1	1	5
2	8	12
3	16	18
4	3	14
5	2	2
6	5	7
7	10	12
8	15	16

- There are 8 jobs
- Drawing jobs on a timeline
- There are 4 distinct solutions
  - [3, 5, 6, 7] and [3, 5, 6, 2]
  - [8, 5, 6, 7] and [8, 5, 6, 2]
- Cannot choose
  - [8, 3, 5, 6, 7] because 8 and 3 overlap

Greedy v0.1 (wrong one)

- Idea: we want maximum number of job, why don't just pick one that does not overlap?
  - Just go through each job, if the job can be selected, pick it
  - From the previous example, we will pick job [1, 2, 3] which is not the best solution

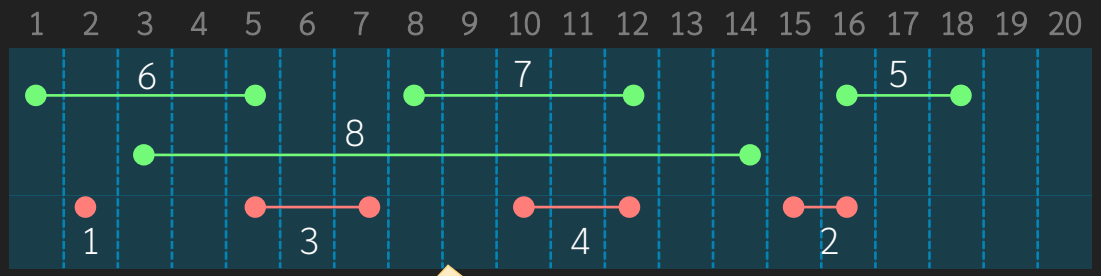


This is written as a recursive program to match the previous code but we can easily convert to looping, that is another exercise

```
def greedy_1(idx,sol)
  if idx == N
    max_length = sol.length
    best = sol
  else
    # check if job #idx conflict with
    # jobs in sol[]
    conflict = false
    for p from 1 to sol.length
      if (intersect(idx,sol[p]))
        conflict = true
        break
    if conflict
      activity(idx+1,sol)
    else
      activity(idx+1,sol.append(idx))
end
```

Greedy v0.2 (another wrong one)

```
def greedy_2()
  jobs = []
  for i in 1 to N
    duration = stop[i] - start[i] +1;
    jobs.append( [duration,start[i],stop[i]] )
  sort(jobs) #sorting using duration
  for i in 1 to N
    start[i] = job[i][2];
    stop[i] = job[i][3];

  # just use greedy v0.1 after sort
  greedy_1(0,[])
end
```

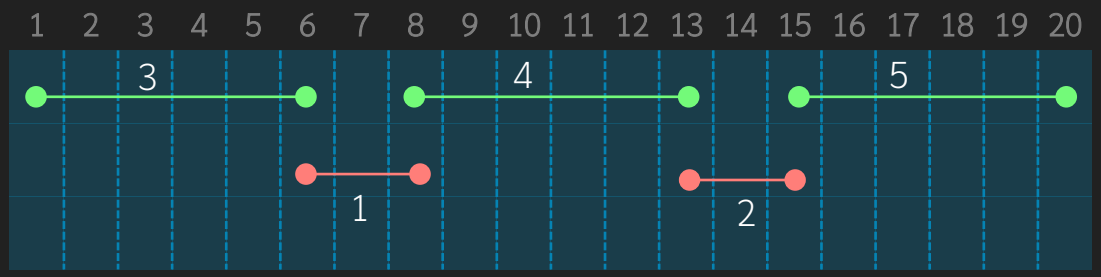


Notice that the index changes. Greedy_2 gives [1, 2, 3, 4] as the output which is correct in this case

- Idea: Don't pick jobs with long duration (because it might conflict with others)
  - Pick short duration first
  - Sort jobs by duration first and use the same logic as v0.1

When v0.2 is wrong?

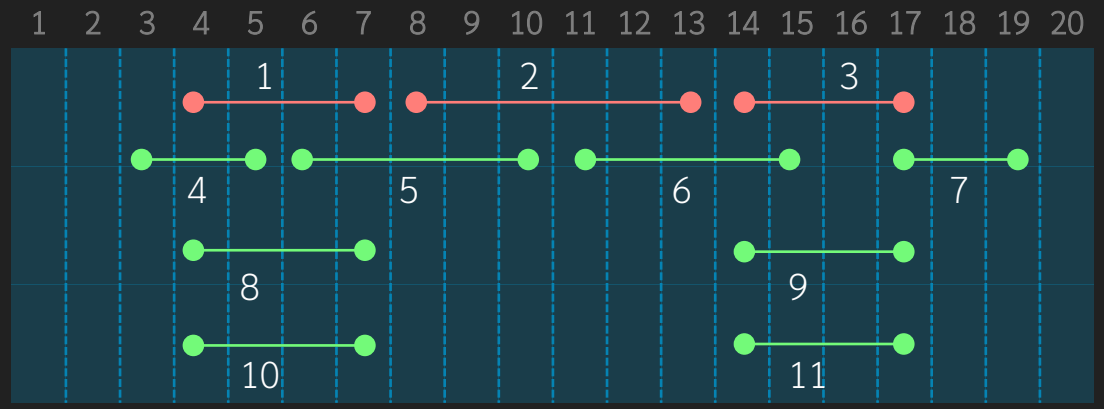
- Shortest jobs do not guarantee best solution
- Example



- Greedy v0.2 gives [1, 2] as the output while the optimum solution is [3, 4, 5]

Greedy v0.3 (yet another wrong one)

- Idea: pick the job that least conflict with other first



#2 has least conflict (overlap with just other two jobs) but if we choose #2, we can choose only one from (1, 8, 10) and another one from (3, 9, 11) which sums as 3 jobs

the best solution is [4, 5, 6, 7] which is 4 jobs

Greedy Actual (the correct one)

- Sort job by stop time and pick one that does not conflict
- Since we consider jobs in increasing value of their stop time, we just keep track of the stop time of our latest job
  - if the start time of the considering job is more than the last stop time, we can include it into our solution
- This is O(n log n), again from sorting
  - The actual selection is O(n)

```
void greedy(vector<int> &sol) {
  //sort by stop time
  vector<pair<int,int>> jobs(n);
  for (int i = 0;i < n;i++)
    jobs[i] = {stop[i],start[i]};
  sort(jobs.begin(),jobs.end());
  for (int i = 0;i < n;i++) {
    start[i] = job[i].second;
    stop[i] = job[i].first;
  }

  // the end time of our latest selected job
  // assume that no job start earlier than time 0
  int last = -1;

  for (int i = 0;i < n;i++) {
    if (start[i] > last) {
      //we can pick job i without conflict
      sol.push_back(i);
      last = stop[i];
    }
  }
}
```

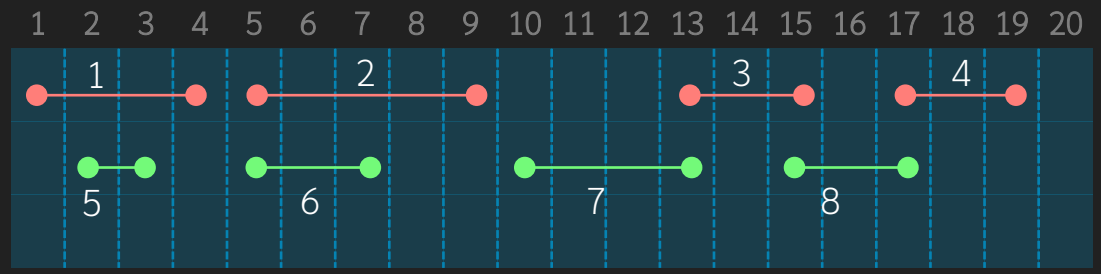
Why Sorting by Stop Time works?

- This needs proof
- We will prove that the solution from our greedy v1 gives the optimum solution
  - Assume that our greedy gives the solution S[1..k]
  - Assume that the best solution is B[1..m]
    - We don’t know what B is and don’t know that k == m
    - We will prove that k == m
    - Assume that B is also sorted by stop time

Proof that Greedy Works

- Proof Idea:
  - We will consider a candidate solution [B[1], B[2], ..., B[m]] and shows that, for i from 1 to n, if S[i] != B[i] we can replace B[i] by S[i] without causing a conflict with other B[ ] and eventually k is equal to m
- Proof by construction
  - First, assume that S and B are sorted by stop time, i.e.,
    - stop[S[i]] < stop[S[i+1]] and stop[B[i]] < stop[B[i+1]]
    - We also knows that stop[S[i]] < start[S[i+1]] and stop[B[i]] < start[B[i+1]]
  - First, consider S[1] and B[1]
    - If S[1] == B[1], we do nothing but if S[1] != B[1] we can replace B[1] by S[1] and the sequence is still not conflict because stop[S[1]] <= stop[B[1]] hence stop[S[1]] < start[B[2]]

B is [1,2,3,4]
S is [5,6,7,8]



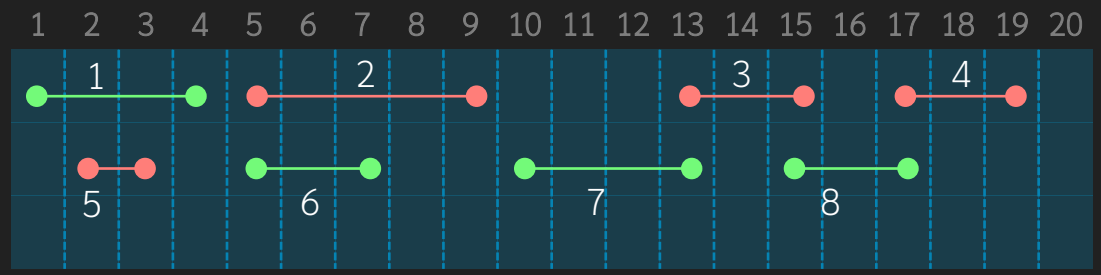
B[1] != S[1], we can replace B[1] by S[1], give [5,2,3,4] which is not conflict because the stop of our solution is guaranteed to be the minimum stop

Proof that Greedy Works

- Now, our solution is [S[1], B[2], B[3], ...,B[m]]. We continue in the same manner from 2 to m
- if S[2] == B[2], we do nothing but if S[2] != B[2] see that following is true
  - stop[S[1]] < start[S[2]] < stop[S[2]] <= stop[B[2]]
- Hence, we can replace B[2] by S[2]
  - Now our solution is [S[1], S[2], B[3], ...,B[m]]

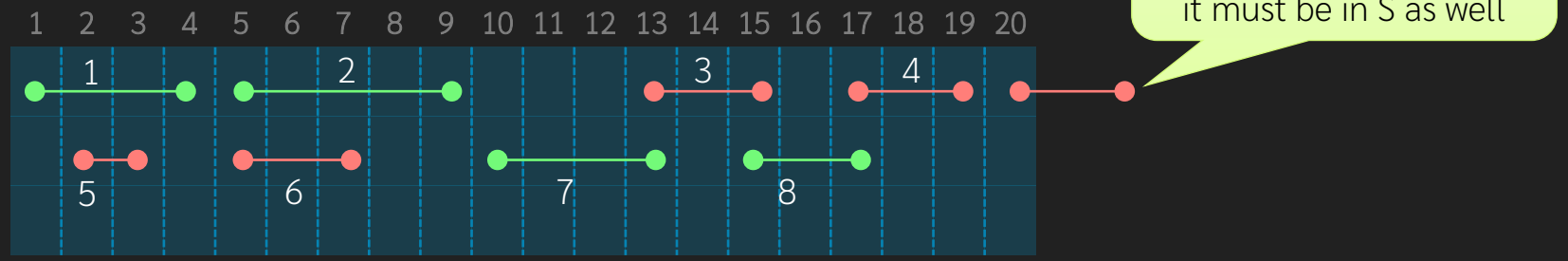
This part is true because we create S by sorting with stop time. If stop[B[2]] is less than stop[S[2]], our algorithm MUST select B[2] instead of S[2]

B is [5,2,3,4]
S is [5,6,7,8]



Proof that Greedy Works

- Keep doing the same from 2 to k
  - Now we have our solution = [S[1], S[2], ..., S[k], B[k+1], B[k+2], ..., B[m]]
- It is not possible to have B[k+1] because if there is it means stop[S[k]] < start[B[k+1]] < stop[B[k+1]] which implies that our algorithm will select B[k+1] into S[k+1]
  - Hence k == m, QED



Summary

- Greedy is choosing each step using simple condition
- We need to prove that our solution produce correct answer